

Parientes de los compiladores

Juancarlos Monsalve Pérez
2023141521
27/8/26

ASM

MACRO

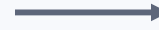
Preprocesador

Ensamblador: traducción cercana a la máquina

- Nació en los años 50 y todavía se usa en contextos donde importa el bajo nivel.
- Traduce lenguaje ensamblador a código máquina u objeto.
- Mantiene eficiencia, pero hace el código más legible que escribir binario puro.
- Influyó en los compiladores porque usa símbolos, pasadas y generación de código.

Antes

```
01001001  
11000011  
...
```



Después

```
LOOP  ADD R1, R2  
      SUB R3, 1  
      JMP LOOP
```

Ventaja principal:
Poco productivo pero muy eficiente

Estructura típica de una línea ensamblador

Una instrucción se organiza en campos fijos.

Etiqueta

Código de operación

Argumentos

Comentario

LOOP

ADD

R1 , R2

; suma valores

- La etiqueta permite nombrar una dirección del programa.
- El código de operación indica qué instrucción ejecutar.
- Los argumentos indican registros, constantes o direcciones.
- Los comentarios ayudan a entender el propósito humano del código.

LC / contador de localidad
Ayuda a saber en qué posición
quedará cada instrucción.

PC / contador de programa
Se relaciona con la
instrucción que se ejecuta
en máquina.

Ensamblador en dos pasadas

La estrategia de dos pasadas resuelve referencias antes de producir el código final.

**1**

Primera pasada: crea la tabla de símbolos y detecta direcciones.

2

Segunda pasada: traduce instrucciones y completa referencias pendientes.

Ejemplo de problema resuelto:
una etiqueta usada antes de ser declarada.

Tabla de símbolos

1

Tabla de símbolos es una estructura que utiliza el ensamblador para guardar información sobre los nombres simbólicos que aparecen en el programa, como etiquetas o nombres definidos por el programador.

2

En los apuntes se relaciona principalmente con el nombre del símbolo y su posición en memoria.

Es primordial, y funciona mediante pasadas
En la primer pasada se crea, y en siguientes pasadas se puede editar según la complejidad del código

		Most Significant Character							
Least Significant Character	Hex	0	1	2	3	4	5	6	7
	0	NUL	DLE	Space	0	@	P	`	p
	1	SOH	DC1	!	1	A	Q	a	q
	2	STX	DC2	*	2	B	R	b	r
	3	ETX	DC3	#	3	C	S	c	s
	4	EOT	DC4	\$	4	D	T	d	t
	5	ENQ	NAK	%	5	E	U	e	u
	6	ACK	SYN	&	6	F	V	f	v
	7	Bell	ETB	'	7	G	W	g	w
	8	BS	CAN	(	8	H	X	h	x
	9	HT	EM	)	9	I	Y	i	y
	A	LF	SUB	*	:	J	Z	j	z
	B	VT	ESC	+	:	K	[	k	{
	C	FF	FS	,	<	L	\	l	
	D	CR	GS	-	=	M	]	m	~
	E	SO	RS	.	>	N	^	n	_
	F	SI	US	/	?	O	_	o	DEL

Macros: abreviar código repetido

- Una macro es una plantilla de texto que el ensamblador expande antes de traducir.
- Sirve para evitar repetir bloques iguales de instrucciones.
- Puede no tener argumentos o recibir parámetros para cambiar partes del bloque.
- El resultado final puede crecer, aunque el código fuente se vea más corto.

Ejemplo mental

```
RUTINA  MACRO
    instrucción 1
    instrucción 2
ENDM
```



RUTINA
se reemplaza por
su cuerpo

Idea clave:
la macro no se “ejecuta” como función; se expande como texto.

Macros con argumentos y anidadas

- Con argumentos, una macro se vuelve reutilizable porque cambia según los valores recibidos.
- Una macro puede llamar a otra macro: eso produce macros anidadas.
- La expansión requiere recordar definiciones y lugares donde se invocan.
- Esto vuelve el ensamblador más potente, pero también más complejo de implementar.

```
SUBROUTINA  MACRO arg1,  
arg2  
    usar arg1  
    usar arg2  
ENDM
```



SUBROUTINA A, B

A y B sustituyen
a arg1 y arg2

Proceso actualizado

1

Guarda dónde se define la macro y su cuerpo.

2

Cuando aparece una llamada, expande o “parchea” el texto correspondiente.

Desensamblador: el camino inverso

- Toma código máquina o binario y lo muestra como instrucciones ensamblador.
- Depende totalmente de la arquitectura: no se desensambla igual en todas las máquinas.
- Puede reconstruir instrucciones, pero pierde nombres originales y parte de la intención humana.
- Por eso genera etiquetas y nombres arbitrarios cuando no conoce los originales.

```
10110000  
01000001  
11101011
```



```
MOV AL, 41h  
JMP label_01
```

Usos comunes

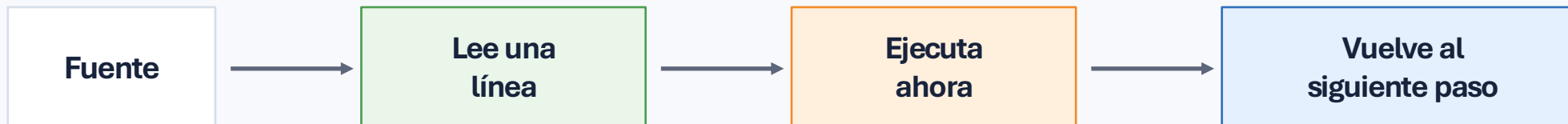
- Ingeniería inversa
- Aprendizaje de instrucciones
- Análisis de BIOS o programas antiguos
- Búsqueda de fallos o parches

Intérpretes: traducen mientras ejecutan

- Combinan traducción con ejecución: leen el programa y van ejecutando paso a paso.
- Normalmente trabajan línea por línea o instrucción por instrucción.
- Dependen más del lenguaje fuente que de generar un archivo objeto completo.
- Su entorno de ejecución acompaña al programa durante todo el proceso.

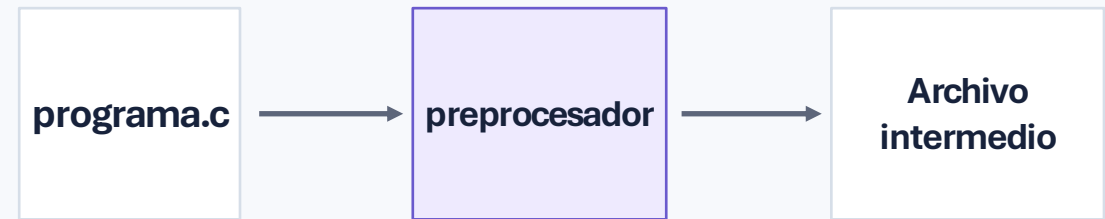
Ejemplos de lenguajes asociados:
Lisp, BASIC, Python, Ruby y Forth.

Ventaja: portabilidad y flexibilidad.
Costo: puede repetir trabajo en ciclos o funciones.



Preprocesador: trabaja antes del compilador

- Actúa antes de la compilación y prepara el archivo fuente.
- En C, reconoce directivas que comienzan con #.
- No revisa toda la semántica del lenguaje C como lo haría el compilador.
- Produce una versión intermedia del programa para que luego se compile.



Funciones típicas:

- incluir archivos
- reemplazar macros
- compilar por condiciones
- manejar directivas especiales

Directivas importantes del preprocesador C

#include

Incluye contenido de otro archivo, como una biblioteca o cabecera.

#define

Define macros o constantes simbólicas para reemplazo textual.

#if / #ifdef

Permite compilación condicional según valores o símbolos definidos.

#elif / #else / #endif

Completa la estructura de condiciones del preprocesador.

#undef

Elimina una definición previa para evitar sustituciones posteriores.

#pragma / #error / #line

Directivas especiales para el compilador, errores o control de líneas.

Caso típico: `#include <stdio.h>` agrega declaraciones necesarias para usar funciones de entrada/salida.

Cuadro Comparativo

Herramienta	Entrada	Proceso central	Salida / resultado
Ensamblador	Código ensamblador	Dos pasadas, símbolos, direcciones	Código máquina u objeto
Macros	Plantillas de texto	Expansión y sustitución	Código fuente expandido
Desensamblador	Código máquina	Traducción inversa por arquitectura	Ensamblador aproximado
Intérprete	Código fuente	Lee, traduce y ejecuta	Resultado inmediato en ejecución
Preprocesador	Fuente con #	Incluye, define y filtra texto	Archivo intermedio

Ideas claves

1

El ensamblador traduce símbolos cercanos al hardware a código máquina.

2

Las macros reducen repetición mediante expansión de texto.

3

El desensamblador intenta reconstruir ensamblador desde binario.

4

El intérprete ejecuta mientras traduce, apoyado en un runtime.

5

El preprocesador prepara el código antes de la compilación.



Fin